

Backend Durum Raporu

Store Operations Platform

Bu rapor, 17 Nisan 2026 itibarıyla backend tarafında yapılan mimari ve uygulama çalışmalarının detaylı özetini, mevcut teknik olgunluk seviyesini ve kademeli yol haritasını sunar.

Proje yönü: NestJS modular monolith + PostgreSQL | Veri sınırları: ops, stg, rpt, audit | Çalışma alanı: backend/nestjs

1. Yönetici Özeti

Çalışmanın ana hedefi, mağaza operasyonu, workforce, checklist, KPI ve reporting süreçlerini taşıyabilecek büyüyebilir bir backend çekirdeği oluşturmaktır. Bu noktada sistem yalnızca CRUD seviyesinde bir API olmaktan çıkmış, veri ingest eden, doğrulayan, materialize eden, snapshot üreten, raporlayan ve operasyonel olarak izlenebilen bir platform omurgasına dönüşmüştür.

Tamamlanan Omurga

Materialization, worker, import observability, reporting read API ve external ID mapping tabanlı integration zinciri çalışır durumda.

Bulunulan Aşama

Backend çekirdeği ileri olgunluk aşamasında. Ana eksen artık altyapı kurmaktan çok işletilebilirlik, güvenilirlik ve kalan bounded slice'ları tamamlama yönünde ilerliyor.

2. Hedeflenen Son Ürün

Nihai hedef, mağaza operasyonlarını taşıyan bir **retail operations backbone** üretmektir. Bu backbone üzerine yönetim paneli, dashboard, bölge müdürü ekranları, denetim akışları ve ileri analitik katmanları oturabilecektir.

- Dış sistemlerden veri alma ve güvenli staging.
- Operational model içine kontrollü materialization.
- Immutable snapshot üretimi ve raporlama.

- Audit izleri ve retry kontrollü async işleme.
- Yeni modül eklemelerinde düzeni bozmayan bounded architecture.

3. Mimari Kararlar ve Korunan Sınırlar

Karar	Anlamı	Neden Kritik
ops	Operasyonel, güncel, çalışan iş verisi	Uygulama davranışını raporlamadan ayırır
stg	Entegrasyonlardan gelen ham ve normalize edilmemiş veri	Kirli dış veri ile domain verisini karıştırmaz
rpt	Immutable snapshot ve raporlama yüzeyi	Raporlamayı operational değişimlerden izole eder
audit	İz, event ve operasyon geçmişi	İşletilebilirlik ve inceleme kabiliyeti sağlar
Worker ayrımı	API process ve async job process ayırır	Ağır işlerin HTTP akışını bozmadan büyümesini sağlar
External ID mapping	stg.external_id_map kanonik eşleme kaynağıdır	Yeni entegrasyonlarda kimlik çözümlemeyi standartlaştırır

4. Bu Çalışmada Neler Yapıldı

4.1 Integration ve Materialization Omurgası

- Materialization validation/error handling sertleştirildi.
- Satır bazlı durum modeli netleştirildi: processed, validation_failed, retryable_error.
- Batch final durumları netleştirildi: completed, completed_with_errors, failed.
- Reference ve dependent entity akışları ayrıştırıldı.
- stg.external_id_map merkezde tutularak mapping lookup/upsert mantığı shared hale getirildi.

4.2 Worker ve Async İşleme

- BullMQ worker host eklendi.
- Worker-only bootstrap eklendi.

- Snapshot üretimi enqueue aşamasından ayrılıp gerçek worker execution semantiğine taşındı.
- `start:workers` script'i eklendi.

4.3 Reporting Katmanı

- Read-only reporting endpoint'leri açıldı.
- Snapshot run, workforce, KPI, checklist ve turnover raporları sağlandı.
- Endpoint'lere pagination ve response metadata eklendi: `count`, `total`, `limit`, `offset`.
- `GET /api/reports/summary` ile dashboard başlangıç özeti üretildi.

4.4 Import Türleri ve Reference Zinciri

- Ayrı import entity desteği eklendi: `assignment`, `position`, `company`, `region`.
- Store, employee, position ve KPI importları mapping-aware hale getirildi.
- KPI importuna scope bazlı dependency guard eklendi.
- Önerilen import sırası kod seviyesinde anlamlı hale geldi: `company` -> `region` -> `store` -> `position` -> `employee` -> `assignment` -> `kpi`.

4.5 Operasyonel Görünürlük ve Retry Kontrolü

- Import batch detail ve error endpoint'leri eklendi.
- Dependency analizi eklendi: `dependencySummary`, `blockedByEntityTypes`, `recommendedImportOrder`.
- Operasyonel karar alanları eklendi: `recommendedNextEntityType`, `canRetryNow`.
- Batch list ve summary endpoint'leri eklendi.
- Controlled retry endpoint'i eklendi.
- Retry tracking alanları eklendi: `retryCount`, `lastRetriedAt`.
- Retry audit event yazımı eklendi: `import_batch.retried`.
- Batch audit geçmişi okunabilir hale getirildi.

5. Şu An Hangi Aşamadayız?

Mevcut aşama: Backend core inşası büyük ölçüde tamamlanmış durumda. Şu an bulunduğumuz seviye, "altyapı var mı?" aşamasından "işletme kalitesi, görünürlük, güvenilirlik ve kalan domain yüzeylerini tamamlama" aşamasına geçmiş durumda.

Alan	Durum	Yorum
Mimari omurga	Güçlü	Schema sınırları, bounded slice yaklaşımı ve async model yerleşti
Integration ingestion	İleri seviye	Mapping-aware, retry kontrollü ve gözlemlenebilir durumda
Reporting API	Kullanılabilir	İlk dashboard ve management ekranlarını besleyebilir
Testing	Temel güvence var	Gerçek integration testleri ve servis testleri mevcut, fakat live infra E2E daha sonra derinleşmeli
Auth / RBAC	Kısmen hazır	Scope mantığı var, enterprise düzey identity entegrasyonu henüz tamamlanmış değil
Snapshot observability	Açık alan	Import kadar zengin operasyonel görünürlük henüz yok
UI readiness	Başlangıç için hazır	Reporting ve import operasyon yüzeyi artık ekran beslemeye elverişli

6. Uygulama Tarafında Şu An Olan Somut Yetenekler

6.1 Reporting API Yüzeyi

- GET /api/reports/snapshot-runs
- GET /api/reports/workforce
- GET /api/reports/kpis
- GET /api/reports/checklists
- GET /api/reports/turnover
- GET /api/reports/summary

6.2 Import Operasyon Yüzeyi

- POST /api/integrations/import-batches
- GET /api/integrations/import-batches
- GET /api/integrations/import-batches/summary
- GET /api/integrations/import-batches/:batchId

- GET /api/integrations/import-batches/:batchId/errors
- POST /api/integrations/import-batches/:batchId/retry
- GET /api/integrations/import-batch-audit/:batchId

6.3 Desteklenen Import Entity Türleri

- company
- region
- store
- position
- employee
- assignment
- kpi

7. Doğrulama ve Mevcut Teknik Güvence

Backend tarafında yaptığımız tüm ana geliştirmelerden sonra test ve build doğrulamaları tekrar çalıştırılarak güncel durum tazelendi.

- **Jest test suite:** `npm test -- --runInBand`
- **Build doğrulaması:** `npm run build`
- Güncel toplam güvence: **8 test suite, 40 test**

8. Güçlü Taraflar ve Kalan Boşluklar

Güçlü Taraflar

- Mimari sınırlar net ve korunuyor.
- Import pipeline artık işletilebilir.
- Reporting katmanı immutable ve okunabilir bir contract sunuyor.
- Yeni modül eklemeye uygun bir düzen oluştu.

Kalan Boşluklar

- Snapshot operasyon görünürlüğü import kadar güçlü değil.
- Auth/RBAC enterprise seviyede tamamlanmadı.
- Canlı PostgreSQL ve Redis ile tam E2E worker doğrulaması genişletilmeli.
- UI tüketimi için bazı summary/health derived contract'lar zenginleştirilebilir.

9. Kademe Kademe Yol Haritası

Kademe 1: Backend Core Kapanışı

Amaç: Import ve snapshot omurgasını tam işletilebilir seviyeye taşımak.

1. Import response'larına derived health state ekleme: healthy, needs_action, blocked, retry_ready.
2. Snapshot ve worker tarafına import benzeri observability yüzeyi açma.
3. Retry ve audit zincirini snapshot run'lar için de standardize etme.
4. Live infra odaklı worker/integration E2E doğrulamalarını artırma.

Kademe 2: Domain Coverage Genişletme

Amaç: Kalan bounded slice'ları aynı mimari disiplinle tamamlamak.

1. Checklist ve turnover tarafında gerekirse ek operational API ve summary yüzeyleri açma.
2. Eksik master/reference entity akışlarını netleştirme.
3. Yeni modüller için aynı pattern'i kullanan reusable application/infrastructure şablonları oluşturma.
4. Domain bazlı audit ve lifecycle event setlerini genişletme.

Kademe 3: Security ve Identity Olgunlaştırma

Amaç: Sistemi gerçek kullanıcı ve organizasyon ortamına yaklaştırmak.

1. Gerçek JWT / identity provider entegrasyonu.
2. Role ve scope matrisi netleştirme.
3. Region/store/company seviyesinde erişim denetimlerini daha sert hale getirme.
4. Audit kayıtlarını güvenlik incelemesi açısından da zenginleştirme.

Kademe 4: Productization ve UI Readiness

Amaç: Backend'i yönetim panelleri ve dashboard'ların doğrudan tüketebileceği seviyeye taşımak.

1. Dashboard için derived summary endpoint'leri tamamlama.
2. Operational admin ekranları için import/snapshot monitoring contract'larını sabitleme.
3. UI ekipleri için response contract dokümantasyonunu finalize etme.
4. Front-end ekranlarının ihtiyaç duyacağı filtre ve sayfalama standardını son haline getirme.

Kademe 5: Scale ve Operasyonel Sertleştirme

Amaç: Sistemi büyüme ve uzun ömürlü kullanım için sağlamlaştırmak.

1. Queue throughput, backoff, retry policy ve idempotency gözden geçirme.
2. Performans ve kapasite testleri.
3. Observability ve alerting standardı.
4. Deployment ve runbook dokümantasyonu.

10. Sonuç

Bugün gelinen noktada backend tarafında dağınık bir başlangıç yapısı değil, büyüyebilir bir platform çekirdeği vardır. Bu çekirdek; integration, materialization, reporting, worker, audit ve test omurgasını aynı mimari disiplin içinde bir araya getirmiştir.

Kısa tanımıyla mevcut durum, "backend iskeleti" aşamasını geçmiş, "operasyonel olarak kullanılabilir backend platformu" aşamasına ulaşmıştır. Bir sonraki kademe, bunu işletme kalitesi daha yüksek, güvenliği daha olgun ve UI katmanına daha doğrudan hizmet eden bir seviyeye taşımaktır.